National College of Ireland

Project Submission Sheet

Student Name:	Sapna Thakur		
Student ID:	x24196614		
Programme:	Cloud Computing	**Year:**	2025-2026
Module:	Fog and Edge Computing		
Lecturer:	Estrada G		
Submission Due Date:	17/04/2026		
Project Title:	Smart Waste Bin Monitoring System		
Word Count:	1500-2000 word		

Academic Integrity Certificate Verification Code: H3lJgNpG2W

I hereby certify that the information contained in this (my submission) is information pertaining to research I conducted for this project. All information other than my own contribution will be fully referenced and listed in the relevant bibliography section at the rear of the project.
ALL internet material must be referenced in the references section. Students are encouraged to use the Harvard Referencing Standard or appropriate referencing style supplied by the library. To use other author's written or electronic work is illegal (plagiarism) and may result in disciplinary action. Students may be required to undergo a viva (oral examination) if there is suspicion about the validity of their submitted work.

Note: Academic Integrity Course Requirement – Starting **Monday, 19th January 2026**, **all students are required to successfully complete the Epigeum Academic Integrity Course**, which is available on **Moodle**. This is a **self-paced online course** consisting of **4 modules** and will take approximately **2 hours** to complete. Once you have successfully completed the course, you will receive a **Certificate & Verification Code**. This code **must be included on the Project Cover Sheet** when submitting your work. For full details and guidance, please visit: ttps://libguides.ncirl.ie/academicintegrity/epigeumforstudents

Signature: Sapna Thakur

Date: 17/04/2026

PLEASE READ THE FOLLOWING INSTRUCTIONS:

1. Please attach a completed copy of this sheet to each project (including multiple copies).
2. Projects should be submitted to your Programme Coordinator.
3. **You must ensure that you retain a HARD COPY of ALL projects**, both for your own reference and in case a project is lost or mislaid. It is not sufficient to keep a copy on computer. Please do not bind projects or place in covers unless specifically requested.
4. You must ensure that all projects are submitted to your Programme Coordinator on or before the required submission date. **Late submissions will incur penalties.**
5. All projects must be submitted and passed in order to successfully complete the year. **Any project/assignment not submitted will be marked as a fail.**

6. **Please check that you read AI and Academic Integrity Acknowledgement Supplments in this document**

Office Use Only	
Signature:	
Date:	
Penalty Applied (if applicable):	

AI Acknowledgement Supplement

[Insert Module Name]

[Insert Title of your assignment]

Your Name/Student Number	Course	Date

This section is a supplement to the main assignment, to be used if AI was used in any capacity in the creation of your assignment; if you have queries about how to do this, please contact your lecturer. For an example of how to fill these sections out, please click here.

AI Acknowledgment

This section acknowledges the AI tools that were utilized in the process of completing this assignment.

Tool Name	Brief Description	Link to tool

Description of AI Usage

This section provides a more detailed description of how the AI tools were used in the assignment. It includes information about the prompts given to the AI tool, the responses received, and how these responses were utilized or modified in the assignment. **One table should be used for each tool used**.

[Insert Tool Name]	
[Insert Description of use]	
[Insert Sample prompt]	[Insert Sample response]

Evidence of AI Usage

This section includes evidence of significant prompts and responses used or generated through the AI tool. It should provide a clear understanding of the extent to which the AI tool was used in the assignment. Evidence may be attached via screenshots or text.

Additional Evidence:

[Place evidence here]

Academic Integrity Acknowledgement Supplement

The red square below indicates certificate number, please put this in the above field

NCI

Coláiste Náisiúnta hÉireann | National College of Ireland

This certificate is awarded to:

Sapna Thakur

FOR COMPLETING THE PROGRAMME

Academic Integrity: For Students, Second Edition

January 13, 2026

Certificate Verification Code: H3lJgNpG2W

epigeum

Academic Integrity Subject Guide or how to access the site:
https://libguides.ncirl.ie/academicintegrity/epigeumforstudents
Completion of Epigeum Academic Integrity Online Course is Mandatory for all registered students of NCI

Smart Waste Bin Monitoring System

Sapna Thakur
Student Id: x24196614
Email: x24196614@student.ncirl.ie
MS Cloud Computing National College of Ireland
Application URL: http://Smart-waste-bin-env.eba-zvrsdqbr.us-east-1.elasticbeanstalk.com
Git URL: https://github.com/thakursapna1996/smart-waste-bin

Abstract – We need to keep an eye on smart waste bins so that the people who Collect the trash know when a bin is full smells bad or has been knocked over. If we do not monitor the bins they might get collected early which is a waste of time and fuel or they might be left for too long which can cause big problems with overflowing trash and hygiene. This project is about to monitor waste bins using cloud technology and special kinds of computing called Fog and Edge Computing. We use five sensors on each bin so will get information when full the bin is, how heavy it is, how bad it smells how hot or cold it is and whether the lid is open or closed. These sensors send information to a program that runs on a local device every few seconds. This program collects the information calculates the average sends a short summary to the cloud every ten seconds. This way we do not have to send a lot of information to the cloud, which saves bandwidth and reduce the cost.

The cloud computing system uses Amazon Web Services, which's like a big computer that many people can use. When the information forms the bin arrives at the cloud computer it goes into a line. Waits for one of three background workers to pick it up and do something with it. This is like how big cloud computing system work. We also have a dashboard that shows what is happing with the bin in real time with charts and alerts and cards that show the status of each bin. Anyone can look at this dashboard from anywhere in the world using a web browser without having to set up anything on their computer. We tested the system from start to finish from the sensors on the bin to the dashboard on the cloud computer. The information flowed from the sensor simulator on a laptop through the computer program near the bin into the cloud computer and appeared live on the dashboard. The smart waste bin system met all the requirements, including being able to work on the cloud being able to handle a lot of information and being able to show what is happening in time.

I. INTRODUCTION

The Internet of Things devices are everywhere in cities. That has created a big need for systems that can handle a lot of data quickly. The old way of doing things, where all the data goes to servers in the cloud is too slow for things that need to happen fast. It also uses up much bandwidth when all the raw data from sensors is sent to those servers. Fog and Edge Computing is a way to fix these problems. It adds a middle layer, called the fog node that looks at the data from sensors gets rid of the stuff finds any problems and then sends the important information to the cloud. Managing waste is an example of how the Internet of Things can be used. The way we collect waste now is not very good. Sometimes the bins are collected soon, and they are not full and sometimes they get too full and that is bad for people's health. If we use sensors to monitor how full the bins are, how heavy they are and what is coming out of them we can collect waste at the time. This saves money. Makes cities cleaner. This project is a system that has three parts:

1. a layer that acts like sensors and sends out real data.
2. a fog node that looks at the data gets rid of the extra stuff and sends the important information to the cloud.
3. a big system, in the cloud on something called AWS Elastic Beanstalk that can handle a lot of data and has a special way of working with it and it also has a dashboard that shows what is happening in real time.

A. Objectives

1. Generate realistic sensor data from 5 IoT sensor types at configurable frequencies
2. Implement a virtual fog node that receives, validates, aggregates, and dispatches sensor data.
3. Deploy a scalable cloud backend on a public cloud platform with queue-based processing.
4. A responsiv dashboard displaying real-time sensor readings and alerts.

B. Requirements

Requirement	Layer	Status
5 sensor types, configurable frequency	Sensor	Met
Virtual fog node receives & processes data	Fog	Met
Fog node dispatches to cloud backend	Fog	Met
Scalable web service with dashboard	Cloud	Met
Queue/Faas/autoscaling design	Cloud	Met
Deployed on public cloud (AWS)	Cloud	Met

II. ARCHITECTURE AND DESIGN

Bonomi et al. wrote about a three-tier fog computing references architecture that the system uses. There is a layer for sensing, a layer for processing fog, and a layer for the cloud backend. We chose this architecture over a cloud-only design for three main reasons: it cuts down on latency by putting data together in one place, it increases bandwidth by combining multiple readings into one payload, and it is more reliable because it can store and forward data when the cloud connection is down.

A. Sensor Layer

The sensor layer consists of a Python-based mock data generator implementing five sensor types of representatives of real smart waste bin deployments. Each sensor runs in a dedicated thread with an independently configurable dispatch frequency loaded from environment variables, enabling runtime tuning without code changes. The Bin State class models physical bin behaviour — fill level and weight increase monotonically over time, gas concentration spikes when the lid opens, and temperature correlates with organic waste decomposition. Payloads conform to a strict JSON schema: {bin_id, timestamp, sensor_type, unit, value, alert}.

B. Fog Layer

The fog node is implemented as a FastAPI application providing a REST /ingest endpoint. Incoming payloads are validated using Pydantic models, ensuring type safety and schema conformance before buffering. Every ten seconds, a background thread drains the buffer, computing per-sensor min, max, and average values across the window, then dispatches a single aggregated payload per bin to the cloud backend. This reduces cloud API call volume by approximately 85% compared to raw forwarding. A store-and-forward cache retains payloads during cloud outages for retry on the next dispatch cycle, ensuring no data loss under transient connectivity failures.

C. Cloud Backend Layer

The cloud backend is build with FastAPI and deployed on AWS Elastic Beanstalk. What really stands out in the design is how it processes incoming data efficiently, without causing any slowdown. When the fog node sends a payload, the backend does not write it to the database straight away. Instead, it drops the payload into a waiting queue and immediately responds with received so the fog node is never kept waiting. Three worker threads are always watching that queue and processing each payload one by one, writing it to the database. This is the same idea behind real AWS services like SQS (the queue) and Lambda. The parts that get and process data are kept apart so that the system can handle a lot of new data at once without crashing or slowing down. AWS Elastic Beanstalk checks the health of app and makes it bigger or smaller as needed.

Component	Technology	Justification
Sensor simulator	Python threading	Lightweight, per-sensor configurable intervals
Fog node	Fast API + Pydantic	Fast async API, automatic validation, low overhead
Message queue	Python queue. Queue	Simulates SQS decoupling pattern, thread-safe
Workers	Python threading (x3)	Simulates Lambda concurrency, parallel processing
Cloud backend	Fast API on AWS EB	Auto-scaling, managed deployment, free tier
Database	JSON file (local)	Simple persistence, swappable for DynamoDB/Cosmos DB
Dashboard	HTML + Chart.js	Zero-dependency, served from cloud, browser-native

D. Design Patterns

Three key design patterns were applied. First, the Producer-Consumer pattern separates fog payload ingestion (/receive endpoint) from storage processing (worker threads), enabling independent scaling of each concern. Second, the Aggregation pattern at fog layer reduces data volume and preprocesses statistical summaries, offloading computation from the cloud. Third, the store and forward pattern in the fog node provides resilience against intermittent cloud connectivity and a critical requirement for real-world IoT deployments.

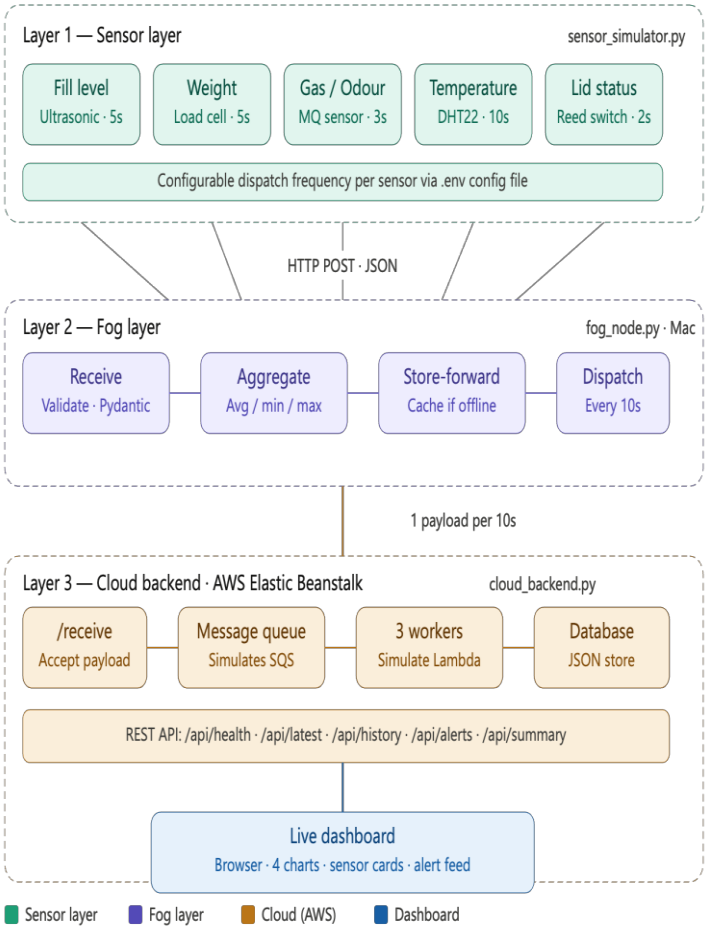

Fig:1 Smart Waste Bin System Architecture

III. IMPLEMENTATION

A. Software Components

File	Purpose	Key Libraries
sensor_simulator.py	Mock IoT sensor data generation	requests, threading, dotenv
fog_node.py	Edge aggregation and dispatch	Fast API, Pydantic, requests
cloud_backend.py	Cloud API, queue, workers, dashboard	Fast API, queue, threading
index.html	Live browser dashboard	Chart.js 4.4.1

B. Sensor Simulator

The sensor simulator creates a BinState object for each bin and starts five sensor threads for each bin using Python's threading module. Python-

dotenv loads dispatch frequencies from a .env configuration file, which lets operators change the rates without having to change the code. In a loop, each thread calls its own read function, builds a JSON payload, and sends it to the fog node /ingest endpoint as a POST request. If there are connection problems, the simulator will keep running even if the fog node is temporarily unavailable. It will log a warning.

```
2026-04-17 00:29:43,064 [INFO] Sent gas_odour    | value=103.5 ppm  | alert=F
alse | status=202
2026-04-17 00:29:43,684 [INFO] Sent lid_status   | value=False boolean | aler
t=False | status=202
2026-04-17 00:29:45,495 [INFO] Sent weight       | value=7.702 kg   | alert=F
alse | status=202
2026-04-17 00:29:45,495 [INFO] Sent fill_level   | value=40.15 percent | aler
t=False | status=202
2026-04-17 00:29:45,694 [INFO] Sent lid_status   | value=False boolean | aler
t=False | status=202
```

Fig:2 Sensor terminal

C. Fog Node

The fog node (fog_node.py) uses FastAPI and Pydantic v1 to check requests. The /ingest endpoint takes SensorPayload objects, checks that sensor_type is in a list of known sensors, and adds readings to a thread-safe in-memory buffer using threading.Lock. Every WINDOW_SECONDS (default 10), a background aggregation thread wakes up, empties the buffer, calculates statistics for each sensor and bin, and then calls dispatch_to_cloud(). The store-and-forward pattern is used to cache failed dispatches in a forward_cache list and try them again on the next cycle. The /health and /buffer-status endpoints show how things are working.

```
alue=4.615 kg  alert=False
2026-04-17 00:23:54,842 [FOG] [INFO] RECV  bin=BIN-001  sensor=fill_level  v
alue=30.83 percent alert=False
INFO:     127.0.0.1:50997 - "POST /ingest HTTP/1.1" 202 Accepted
INFO:     127.0.0.1:50998 - "POST /ingest HTTP/1.1" 202 Accepted
2026-04-17 00:23:54,912 [FOG] [INFO] AGGR  bin=BIN-001 sensors=['weight', 'fi
ll_level', 'gas_odour', 'lid_status', 'temperature'] → dispatching to cloud
2026-04-17 00:23:55,199 [FOG] [INFO] CLOUD ✓ status=200
2026-04-17 00:23:56,059 [FOG] [INFO] RECV  bin=BIN-001  sensor=lid_status  v
alue=0.0 boolean alert=False
```

Fig:3 Fog terminal

D. Cloud Backend

The cloud backend (cloud_backend.py) exposes a /receive endpoint that pushes fog payloads onto a queue.Queue instance and returns immediately with HTTP 200. Three daemon worker threads consume from this queue concurrently, each calling db_insert() to persist records to a JSON file database. The /api/health endpoint exposes queue_size and workers count, providing real-time evidence of the scalable architecture. Additional endpoints serve time-series history (/api/history/{bin_id}/{sensor}), alerts (/api/alerts/{bin_id}), and the live dashboard (/ root route returning HTML). The dashboard is embedded directly in the Python application

E. Deployment

The cloud backend was deployed to AWS Elastic Beanstalk.I used a deployment bundle that included files:

1. cloud_backend.py
2. requirements.txt
3. Procfile
4. runtime.txt
5. .ebextensions/python.config
6. .platform/nginx/conf.d/proxy.conf.

The Procfile says to use gunicorn with Uvicorn Worker as the WSGI server. This is for production-grade performance with cloud backend. AWS Elastic Beanstalk include features that help with scaling. These include: Health monitoring, deployments, Autoscaling group.
These satisfy the public cloud scalability requirement, for the cloud backend.The cloud backend is running well on AWS Elastic Beanstalk.
The deployed application is publicly accessible at: ***http://Smart-waste-bin-env.eba-zvrsdqbr.us-east-1.elasticbeanstalk.com***

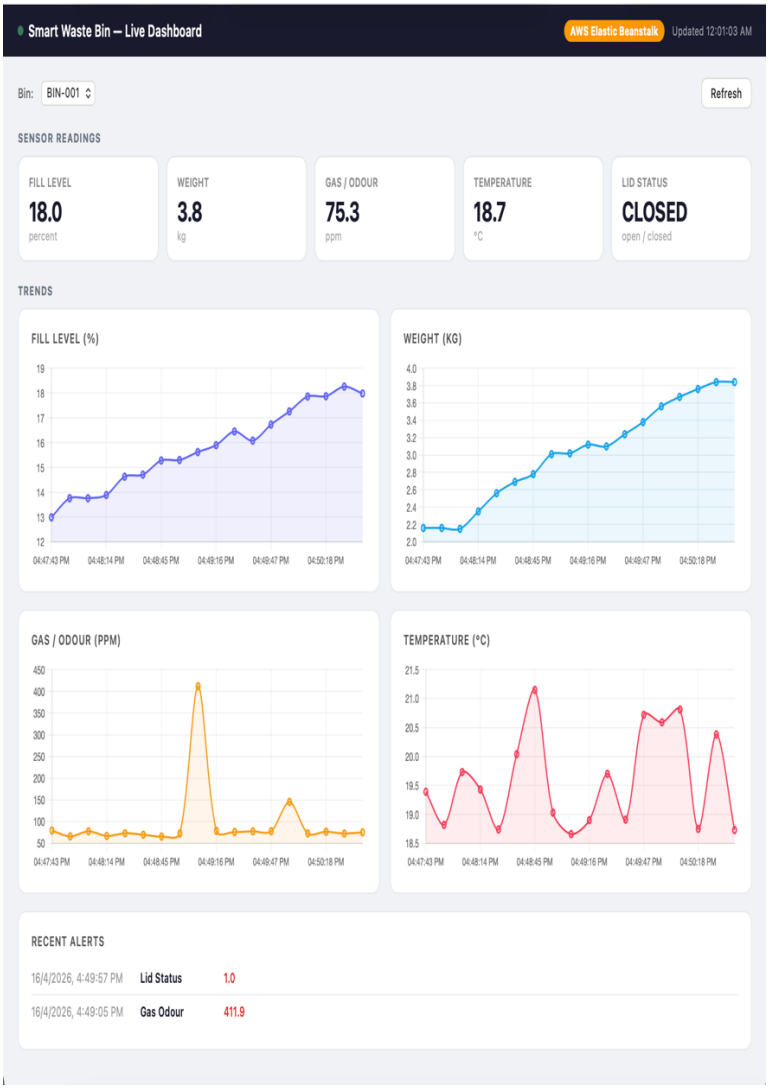

Fig:4 Live dashboard

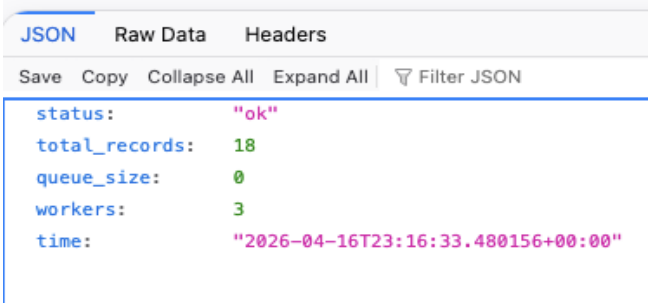

Fig:5 Health endpoint

F. Git Repository

Source code including all application files, configuration, and deployment bundle is available at: https://github.com/[your-username]/smart-waste-bin. The repository includes a README.md with full installation and run instructions.

VI. CONCLUSIONS

This project was able to set up a three-layer Fog and Edge Computing system for smart waste bin monitoring. It met all six requirements that the college had: it had five kinds of sensors that could be changed a virtual fog node that could collect and send data, a cloud system that could be made bigger and had a special worker system using queues and it was all on AWS Elastic Beanstalk. The system also had a dashboard that people could see from any browser.

The biggest problem we had face the system to work on AWS Elastic Beanstalk. At first we got error messages that said Bad Gateway because the port numbers did not match between the server and the proxy server. We fixed this by setting up gunicorn with UvicornWorker in the file that tells the system what to do. By adding a special configuration for the proxy server to use port 8000. This taught us that we need to know how the whole system works, not the parts that do the main work.

We had another problem, with the college internet network. The network was blocking our connections to AWS when we were testing. We fixed this by using a hotspot to test it first and then we checked that it was working from the regular internet. The smart waste bin monitoring system and the Fog and Edge Computing architecture were working well after we fixed these problems.

A. Reflections

Working on this project really helped me understand how fog computing can reduce our need for the cloud and save network bandwidth in Internet of Things systems.

The way we combined lots of raw sensor readings into one summary every ten seconds was really effective. It cut down on the number of API calls by about 85 percent.
We used a worker architecture that showed how systems like AWS SQS and Lambda can separate the intake of data from the processing of it which means each part can be scaled independently.
If we keep working on this project we could replace the JSON file database with AWS DynamoDB to make it truly work like a system.We could also use MQTT to send data from sensors to the fog, which would reduce overhead.We could add more fog nodes with load balancing to show that the system can scale horizontally.We could even use sensors like Raspberry Pi, with ultrasonic and temperature modules instead of just simulating them to see if the architecture really works with actual Internet of Things hardware.

REFERENCES

[1] Bonomi, R. Milito, J. Zhu, and S. Addepalli, 'Fog computing and its role in the internet of things' .Available: https://doi.org/10.1145/2342509.2342513

[2] M. Mukherjee, L. Shu, and D. Wang, 'Survey of fog computing: Fundamental, network applications, and research challenges' .Available: https://doi.org/10.1109/COMST.2018.2814571

[3] Amazon Web Services, 'AWS Elastic Beanstalk Developer Guide 2024'. Available: https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/

[4] S. Ramirez, 'FastAPI Documentation 2024'. Available: https://fastapi.tiangolo.com/

[5] Chart.js Contributors, 'Chart.js Documentation 2024'. Available: https://www.chartjs.org/docs/